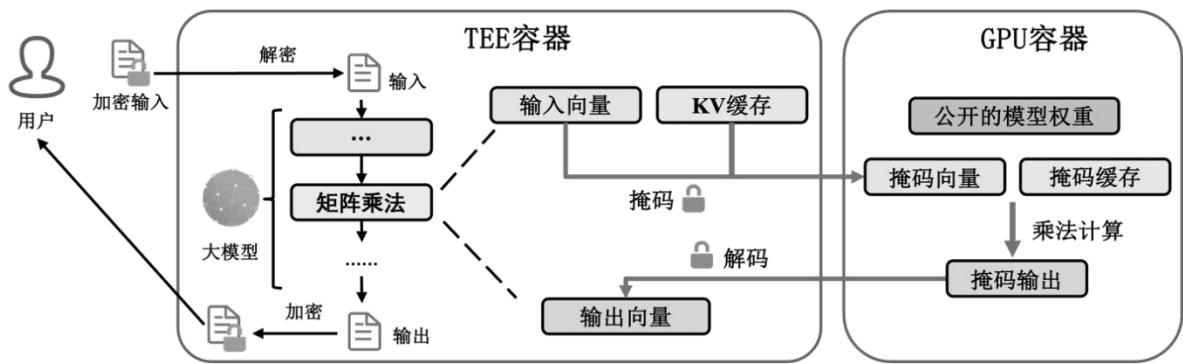


说明书摘要

本发明属于计算信息安全技术领域，具体为在计算系统中保护数据隐私的矩阵乘法加速计算方法。本发明方法包括：在可信执行环境中，获取一或多个私密输入数据，生成随机掩码，利用随机掩码对私密输入数据进行计算生成掩码后的输入数据；掩码后的输入数据从可信执行环境发送至 GPU 计算加速器；GPU 计算加速器对掩码后的输入数据执行核心计算，生成掩码后的结果；将掩码后的结果从 GPU 计算加速器发送回可信执行环境；可信执行环境对掩码后的结果进行解码运算，恢复出基于所述私密输入数据的最终计算结果。本发明在利用 GPU 强大算力加速大模型推理的同时，有效保障用户数据的隐私性，为在不可信云环境中部署隐私保护 AI 服务提供技术支撑。

摘要附图



权利要求书

1. 一种在计算系统中保护数据隐私的矩阵乘法加速计算方法，其特征在于，允许在可信执行环境（TEE）中运行的应用程序，能够将其内部的矩阵乘法安全地卸载到 TEE 外部不可信的 GPU 上执行，在不泄露私密数据的前提下，实现对大语言模型推理任务的计算加速；其中包括：

（1）**掩码处理**：在数据离开可信执行环境（TDX）前，先使用一个或多个在 TDX 内部秘密生成的随机向量或随机矩阵，统称为“掩码”，对原始私密数据进行“掩码”处理，即原始私密数据加上该“掩码”；

（2）**卸载计算**：将经过掩码处理的数据发送给不可信的 GPU；

（3）**非可信计算**：GPU 在掩码后的数据上执行矩阵乘法，得到同样是掩码状态的结果；

（4）**恢复**：GPU 将计算结果返回给 TDX；TDX 利用其内部保存的原始掩码信息，通过一系列代数运算，从掩码的结果中减去或抵消掉掩码所带来的影响，最终精确地恢复出直接在原始私密数据上进行运算本应得到的正确结果；

具体步骤为：

（一）在所述可信执行环境中，获取一或多个私密输入数据；

（二）在所述可信执行环境中，生成至少一个随机掩码；

（三）在所述可信执行环境中，利用所述随机掩码对所述私密输入数据进行计算，生成掩码后的输入数据；

这里，所述私密输入数据，是一私密输入向量 $\vec{x}$ ，或者是第一私密矩阵 K 和第二私密矩阵 Q ，或者是一私密查询向量 $\vec{q}$ 和一动态增长的私密键缓存矩阵 ΔK ；

（四）将所述掩码后的输入数据从所述可信执行环境发送至所述不可信 GPU 计算加速器；

（五）由所述不可信 GPU 计算加速器对所述掩码后的输入数据执行预定的核心计算，生成一掩码后的结果；

（六）将所述掩码后的结果从所述不可信 GPU 计算加速器发送回所述可信执行环境；

（七）在所述可信执行环境中，对所述掩码后的结果进行解码运算，以移除所述随机掩码对核心计算结果造成的影响，从而恢复出基于所述私密输入数据的最终计算结果。

2. 根据权利要求 1 所述的矩阵乘法加速计算方法，其特征在于，所述私密输入数据为一私密输入向量 $\vec{x}$ ，于是：

所述随机掩码为一与所述私密输入向量 $\vec{x}$ 同维度的随机向量 $\vec{r}$ ；

所述利用随机掩码对所述私密输入数据进行计算，生成掩码后的输入数据，具体是将所述私密输入向量与所述随机向量相加，得到掩码后的输入向量： $\vec{x'} = \vec{x} + \vec{r}$ ；

所述核心计算为掩码后的输入向量 $\vec{x'}$ 与一公开权重矩阵 W 进行向量与矩阵的乘法运算；其中，所述公开权重矩阵 W 是开源大语言模型的全连接层参数，被视为公开信息，并可预先加载至不可信 GPU 计算加速器的内存中；

进一步，进行校正运算，包括：从所述不可信 GPU 计算加速器返回的掩码后结果 $\vec{y'}$ 中，减去一个预先计算的校正项 $\vec{c}$ ；所述校正项 $\vec{c}$ 是所述随机向量 $\vec{r}$ 与所述公开权重矩阵 W 的乘积： $\vec{c} = \vec{r} \cdot W$ 。

3. 根据权利要求 1 所述的矩阵乘法加速计算方法，其特征在于，所述私密输入数据为第二私密矩阵 Q 和第一私密矩阵 K ，于是：

所述随机掩码为与第一私密矩阵 K 和第二私密矩阵 Q 相同维度的第一掩码矩阵 M_1 和第二掩码矩阵 M_2 ；

所述利用随机掩码对所述私密输入数据进行计算，生成掩码后的输入数据，具体是将所述第一私密矩阵 K 与所述第一掩码矩阵 M_1 相加得到掩码后的第一矩阵 $K' = K + M_1$ ；，并将所述第二私密矩阵 Q 与所述第二掩码矩阵 M_2 相加得到掩码后的第二矩阵 $Q' = Q + M_2$

所述核心计算为所述第二矩阵 Q' 乘上第一矩阵 K' 的转置矩阵： $Q' \cdot K'^T$ ；

并进行校正运算，即从所述不可信 GPU 计算加速器返回的掩码后结果 S' 中，减去所述第二私密矩阵 Q 与所述第一掩码矩阵 M_1 转置的乘积、所述第二掩码矩阵 M_2 与所述第一私密矩阵 K 转置的乘积，以及所述第二掩码矩阵 M_2 与所述第一掩码矩阵 M_1 转置的乘积：

$$S = S' - Q \cdot M_1^T - M_2 \cdot K^T - M_2 * M_1^T = S' - Q \cdot U_1^T \cdot V_1^T - U_2 \cdot (V_2 \cdot K^T) - M_2 * M_1^T；$$

其中，所述第一掩码矩阵 M_1 和第二掩码矩阵 M_2 均为低秩矩阵，分别由两个更小的随机矩阵相乘生成： $M_1 = U_1 \cdot V_1$ ， $M_2 = U_2 \cdot V_2$ ，其中所述低秩矩阵的秩 r 通常设为十以内的正整数，以减少计算量。

4. 根据权利要求 1 所述的矩阵乘法加速计算方法，其特征在于，所述私密输入数据为一私密查询向量 $\vec{q}$ 、一个历史键缓存 K 和一个掩码后的历史键缓存 K' ，于是：

(1) TEE 进行预计算，即利用一个基于所述固定随机矩阵 $\overline{H}$ 和历史键缓存 K 预先计算并重复使用的校正项 $C_h = H \cdot K^T$ ，以降低在线计算开销；

(2) 对每一个新的私密查询向量，TEE 使用一个新的一次性随机向量 $\vec{q}$ 和一个固定的随机矩阵 H 生成查询掩码，得到掩码后的查询向量： $\vec{q}' = \vec{q} + \vec{g} \cdot H$ ；

(3) GPU 计算加速器进行核心计算，为所述掩码后的查询向量 $\vec{q}'$ 与所述一个掩码后的历史键缓存 K' 的转置相乘： $\vec{s}' = \vec{q}' \cdot K'^T$ ；

(4) TEE 进行校正运算：

$$\vec{s} = \vec{s}' - \vec{q}' * M_1 - \vec{g} \cdot C_h$$

5. 根据权利要求 4 所述的矩阵乘法加速计算方法，其特征在于，所述方法还包括一个动态更新所述掩码键缓存的方法，其步骤为：

(1) 累积与触发：可信执行环境（TEE）在其内部安全内存中开辟一个缓冲区，用于累积自上次更新以来新生成的键向量；这些键向量被逐行堆叠，形成一个更新矩阵 ΔK ；当该矩阵的行数达到预设的阈值时，批量更新流程被自动触发；

(2) 掩码生成与处理：一旦更新流程被触发，TEE 生成一套全新的、用于此次更新的低秩随机掩码 ΔM_k ；该掩码同样由两个更小的随机矩阵相乘生成；然后，TEE 在内部计算掩码后的更新矩阵： $\Delta K' = \Delta K + \Delta M_k$ ；

(3) 卸载与合并：TEE 将计算出的掩码后更新矩阵 $\Delta K'$ 发送至不可信的 GPU；GPU 接收到数据后，将其在行维度上拼接到已存储的掩码键缓存 K' 的末尾，形成新的、尺寸更大的掩码键缓存： $K' = \text{concat}(K', \Delta K')$ ；

(4) 矫正项同步：TEE 更新相关的校正项 $C_h = H \cdot K^T + H \cdot \Delta K$ ，以确保后续新的查询请求能够获得正确的计算结果。

说明书

在计算系统中保护数据隐私的矩阵乘法加速计算方法

技术领域

本发明属于计算信息安全技术领域，具体涉及大模型推理过程中的矩阵乘法加速计算方法。

背景技术

随着大语言模型（LLM）的广泛应用，其推理服务中的用户隐私保护问题变得至关重要。例如，用户可能会输入个人身份信息、财务状况、医疗记录等敏感数据。为了保护这些数据不被包括云服务提供商在内的任何非授权方访问，可以采用可信执行环境（Trusted Execution Environment, TEE）技术，如 Intel 的 Trust Domain Extensions (TDX)。TDX 技术通过 CPU 硬件特性创建出一个被称为“信任域”（Trust Domain）的内存隔离区域，在该区域内运行的代码和处理的数据能够得到硬件级别的保护，即使是操作系统、虚拟机管理等更高权限的软件也无法窥探或篡改其内容。

然而，TDX 这类可信执行环境所能提供的计算资源（如 CPU 核心数、内存大小等）通常是有限的，难以高效的处理大模型推理过程中的矩阵乘法。为了获得可以接受的响应速度，通常需要使用图形处理器（GPU）这类拥有数千个并行处理核心的硬件进行加速。

这就产生了一个核心的技术矛盾：一方面，用户数据必须在受保护的 TDX 环境中处理以保障隐私；另一方面，为了加速乘法计算，又需要将乘法计算任务卸载到 TDX 环境之外的、性能强大但被视为不可信的 GPU 上。直接将未经处理的私密数据发送给 GPU 会造成隐私泄露。因此，如何设计一种高效的矩阵乘法计算卸载机制，既能利用 GPU 的强大并行计算能力，又能确保用户数据在整个计算过程中不被泄露，是当前在不可信云环境中部署隐私保护大模型推理服务所面临的关键挑战。

发明内容

本发明旨在解决上述背景技术中提出的问题，提供一种在计算系统中保护数据隐私的矩阵乘法加速计算方法，

本发明提供的在计算系统中保护数据隐私的矩阵乘法加速计算方法，是允许在可信执行环境（TEE）中运行的应用程序，能够将其内部的矩阵乘法安全地卸载到 TEE 外部不可信的 GPU 上执行，从而在不泄露私密数据的前提下，实现对大语言模型推理等任务的计算加速；其核心内容为：

（1）**掩码（Masking）处理**：在数据离开可信执行环境（TDX）前，先使用一个或多个在 TDX 内部秘密生成的随机向量或随机矩阵（统称为“掩码”）对原始的私密数据进行“掩码”处理。这种掩码是通过加法操作完成的。

（2）**卸载计算（Offloading）**：将经过掩码处理的数据发送给不可信的 GPU。

（3）**非可信计算（Untrusted Computation）**：GPU 在掩码后的数据上执行矩阵乘法，并得到一个同样是掩码状态的结果。

（4）**恢复（Recovery）**：GPU 将计算结果返回给 TDX；TDX 利用其内部保存的原始掩码信息，通过一系列的代数运算，从掩码的结果中减去或抵消掉掩码所带来的影响，最终精确地恢复出：如果直接在原始私密数据上进行运算本应得到的正确结果。

由于 GPU 接触到的所有数据都是经过随机掩码处理的，其作用类似于密码学中的“一次性密码本”，使得 GPU 难以从掩码数据中反推出关于原始私密数据的有用信息，从而保障了隐私。

具体地，在计算系统中保护数据隐私的矩阵乘法加速计算方法，所述计算系统包括一可信执行环境（TEE）和一在该可信执行环境外的不可信 GPU 计算加速器，具体步骤为：

（一）在所述可信执行环境中，获取一或多个私密输入数据；

（二）在所述可信执行环境中，生成至少一个随机掩码；

（三）在所述可信执行环境中，利用所述随机掩码对所述私密输入数据进行计算，生成掩码后的输入数据；

这里，所述私密输入数据，是一私密输入向量 $\vec{x}$ ，或者是第一私密矩阵 K 和第二私密矩阵 Q ，或者是一私密查询向量 $\vec{q}$ 和一动态增长的私密键缓存矩阵 ΔK ；

（四）将所述掩码后的输入数据从所述可信执行环境发送至所述不可信 GPU 计算加速器；

（五）由所述不可信 GPU 计算加速器对所述掩码后的输入数据执行预定的核心计算，生成一掩码后的结果；

（六）将所述掩码后的结果从所述不可信 GPU 计算加速器发送回所述可信执行环境；

(七) 在所述可信执行环境中, 对所述掩码后的结果进行解码运算, 以移除所述随机掩码对核心计算结果造成的影响, 从而恢复出基于所述私密输入数据的最终计算结果。

进一步地:

(1) 所述私密输入数据为一私密输入向量 $\vec{x}$, 于是:

所述随机掩码为一与所述私密输入向量 $\vec{x}$ 同维度的随机向量 $\vec{r}$;

所述利用随机掩码对所述私密输入数据进行计算, 生成掩码后的输入数据, 具体是将所述私密输入向量与所述随机向量相加, 得到掩码后的输入向量: $\vec{x}' = \vec{x} + \vec{r}$;

所述核心计算为掩码后的输入向量 $\vec{x}'$ 与一公开权重矩阵 W 进行向量与矩阵的乘法运算; 所述公开权重矩阵 W 是大语言模型全连接层的固有参数, 被视为公开信息, 并可预先加载至不可信 GPU 计算加速器的内存中;

进一步, 进行校正运算, 包括: 从所述不可信 GPU 计算加速器返回的掩码后结果 $\vec{y}$ 中, 减去一个预先计算的校正项 $\vec{c}$; 所述校正项 $\vec{c}$ 是所述随机向量 $\vec{r}$ 与所述公开权重矩阵 W 的乘积: $\vec{c} = \vec{r} \cdot W$ 。

(2) 所述私密输入数据为第一私密矩阵 K 和第二私密矩阵 Q , 于是:

所述随机掩码为与第一私密矩阵 K 和第二私密矩阵 Q 相同维度的第一掩码矩阵 M_1 和第二掩码矩阵 M_2 ;

所述利用随机掩码对所述私密输入数据进行计算, 生成掩码后的输入数据, 具体是将所述第二私密矩阵 Q 与所述第二掩码矩阵 M_2 相加得到掩码后的第一矩阵 $Q' = Q + M_2$, 并将所述第一私密矩阵 K 与所述第一掩码矩阵 M_1 相加得到掩码后的第一矩阵 $K' = K + M_1$;

所述核心计算为所述第二矩阵 Q' 乘上第一矩阵 K' 的转置矩阵: $Q' \cdot K'^T$;

并进行校正运算, 即从所述不可信 GPU 计算加速器返回的掩码后结果 S' 中, 减去所述第二私密矩阵 Q 与所述第一掩码矩阵 M_1 转置的乘积、所述第二掩码矩阵 M_2 与所述第一私密矩阵 K 转置的乘积, 以及所述第二掩码矩阵 M_2 与所述第一掩码矩阵 M_1 转置的乘积:

$$S = S' - Q \cdot M_1^T - M_2 \cdot K^T - M_2 * M_1^T = S' - Q \cdot U_1^T \cdot V_1^T - U_2 \cdot (V_2 \cdot K^T) - M_2 * M_1^T。$$

其中, 所述第一掩码矩阵 M_1 和第二掩码矩阵 M_2 均为低秩矩阵, 分别由两个更小的随机矩阵相乘生成: $M_1 = U_1 \cdot V_1$, $M_2 = U_2 \cdot V_2$, 其中所述低秩矩阵的秩 r 通常设为十以内的正整数, 以减少计算量。

(3) 所述私密输入数据为一私密查询向量 $\vec{q}$ 、一个历史键缓存 K 和一个掩码后的历史键缓存 K' ，于是：

(1) TEE 进行预计算，即利用一个基于所述固定随机矩阵 H 和历史键缓存 K 预先计算并重复使用的校正项 $C_h = H \cdot K^T$ ，以降低在线计算开销。

(2) 对每一个新的私密查询向量，TEE 使用一个新的一次性随机向量 $\vec{g}$ 和一个固定的随机矩阵 H 生成查询掩码，加到私密查询向量 $\vec{q}$ 后，得到掩码后的查询向量： $\vec{q'} = \vec{q} + \vec{g} \cdot H$ ；

(3) GPU 计算加速器进行核心计算，为所述掩码后的查询向量 $\vec{q'}$ 与所述一个掩码后的历史键缓存 K' 的转置相乘： $\vec{s'} = \vec{q'} \cdot K'^T$ （等于 $(\vec{q} + \vec{g} \cdot H) \cdot (K^T + M_1)$ ）；

(4) TEE 进行校正运算：

$$\vec{s} = \vec{s'} - \vec{q'} \cdot M_1 - \vec{g} \cdot C_h = \vec{s'} - \vec{q'} \cdot U_1 \cdot V_1 - \vec{g} \cdot C_h$$

所述方法还包括一个动态更新所述掩码键缓存的方法，包括：

(1) 累积与触发：可信执行环境（TEE）在其内部安全内存中开辟一个缓冲区，用于累积自上次更新以来新生成的键向量。这些键向量被逐行堆叠，形成一个更新矩阵 ΔK 。当该矩阵的行数（即累积的 Token 数量）达到预设的阈值（例如 128）时，批量更新流程被自动触发。

(2) 掩码生成与处理：一旦更新流程被触发，TEE 会生成一套全新的、用于此次更新的低秩随机掩码 ΔM_k 。为了保持计算高效性，该掩码同样由两个更小的随机矩阵相乘生成（ $\Delta M_k = \Delta U_k * \Delta V_k$ ）。随后，TEE 在内部计算掩码后的更新矩阵： $\Delta K' = \Delta K + \Delta M_k$ 。

(3) 卸载与合并：TEE 将计算出的掩码后更新矩阵 $\Delta K'$ 发送至不可信的 GPU。GPU 接收到数据后，将其在行维度上（即 Token 维度）拼接到已存储的掩码键缓存 K' 的末尾，形成新的、尺寸更大的掩码键缓存： $K' = \text{concat}(K', \Delta K')$ 。

(4) 矫正项同步：TEE 更新相关的校正项（ $C_h = H \cdot K^T + H \cdot \Delta K$ ），以确保后续新的查询请求能够获得正确的计算结果。

与现有技术相比，本发明具有以下有益效果：

(1) **强隐私保障：**通过加法掩码机制，确保了原始私密数据不出可信执行环境，GPU 等不可信设备无法访问用户的真实输入，提供了强大的数据机密性保证。

(2) 显著的性能提升： 成功地将计算复杂度最高的核心运算转移到高性能 GPU 上执行，而 TDX 中仅保留了计算复杂度较低的掩码生成、数据加减和结果校正等操作，极大地提升了整个系统的运算性能和响应速度。

(3) 方案的适应性： 针对大语言模型推理中常见的不同类型的矩阵乘法运算，如私密输入与公开权重的乘法、两个私密矩阵的乘法，以及私密向量与动态私密矩阵的乘法，本发明提供了三种针对性的优化方案，覆盖了关键的计算场景。

附图说明

图 1 为基于可信执行环境和 GPU 的高效矩阵乘法加速计算方法流程图。

具体实施方式

为使本发明的目的、技术方案和优点更加清楚，下面将详细描述本发明的具体实施方式。在此，可信执行环境（TEE）以 Intel TDX 为例，不可信计算加速器以 GPU 为例进行说明，但不限于此。

实施例 1：私密输入与公开权重的乘法

针对大语言模型中常见的全连接层或线性投影运算，例如 $\vec{y} = \vec{x} \cdot W$ ，其中输入向量 $\vec{x} \in \mathbb{R}^{1 \times d}$ （ $\mathbb{R}^{1 \times d}$ 代表维度为 $1 \times d$ 且内部元素为实数的向量的集合）是用户私密数据，而权重矩阵 $W \in \mathbb{R}^{d \times m}$ （ $\mathbb{R}^{d \times m}$ 代表维度为 $d \times m$ 且内部元素为实数的矩阵的集合，以下同理）是开源大模型的模型参数。

具体步骤如下：

(1) 离线阶段（TDX 内执行）：

TDX 生成一个与输入向量 $\vec{x}$ 同维度的随机向量 $\vec{r} \in \mathbb{R}^{1 \times d}$ 。

TDX 利用已知的公开权重 W 预先计算一个校正项 $\vec{c} = \vec{r} \cdot W \in \mathbb{R}^{1 \times d}$ 。此步骤可在用户请求到达前完成。

(2) 在线阶段（数据从 TDX 发往 GPU）：

(a) TDX 接收到用户的私密输入向量 $\vec{x}$ 。

(b) TDX 计算掩码后的输入： $\vec{x}' = \vec{x} + \vec{r}$ 。

(c) TDX 将掩码向量 $\vec{x}'$ 发送给 GPU。权重矩阵 W 可预先存放在 GPU 显存中或由 GPU 直接访问。

(3) GPU 计算阶段：

(a) GPU 接收到 $\vec{x}'$ ，执行核心的向量-矩阵乘法运算： $\vec{y}' = \vec{x}' \cdot W$ 。

(b) GPU 将计算出的掩码结果 $\vec{y}'$ 发送回 TDX。

(4) 最终计算阶段（TDX 内执行）：

(a) TDX 接收到 $\vec{y}'$ ，使用离线阶段预计算的校正项 $\vec{c}$ 来恢复最终结果： $\vec{y} = \vec{y}' - \vec{c}$ 。

分析： 在此方案中，TDX 的在线运行时开销仅为一次向量加法（ $O(d)$ ）和一次向量减法（ $O(m)$ ），计算量极低。而计算复杂度高达 $O(dm)$ 的核心乘法被成功卸载到 GPU。由于随机向量 $\vec{r}$ 对 GPU 是保密的，GPU 难以从 $\vec{x}'$ 中分离出 $\vec{x}$ 的信息，安全性得到保障。

实施例 2：两个私密矩阵的乘法

针对自注意力机制中的核心计算 $S = Q \cdot K^T$ ，其中查询矩阵 $Q \in \mathbb{R}^{n \times d}$ 和键矩阵 $K \in \mathbb{R}^{n \times d}$ 都源于用户输入，因此都是私密的。

具体步骤如下：

(1) 离线阶段（TDX 内执行）：

(a) TDX 生成四个较小的随机矩阵：

$$U_1 \in \mathbb{R}^{n \times r}, V_1 \in \mathbb{R}^{r \times d}, U_2 \in \mathbb{R}^{n \times r}, V \in \mathbb{R}^{r \times d},$$

其中参数 r 通常设为一个十以内的正整数，远小于 n 和 d （通常为一千以上）。

(b) TDX 通过这些小矩阵生成两个低秩的掩码矩阵：

$$M_1 = U_1 \cdot V_1 \in \mathbb{R}^{n \times d} \text{ 和 } M_2 = U_2 \cdot V \in \mathbb{R}^{n \times d}。$$

TDX 预计算校正项 $C = M_2 \cdot M_1^T$ 。

(2) 在线阶段（数据从 TDX 发往 GPU）：

(a) TDX 获得私密矩阵 Q 和 K 。

(b) TDX 计算掩码后的矩阵： $Q' = Q + M_2$ 和 $K' = K + M_1$ 。

(c) TDX 将 Q' 和 K' 发送给 GPU。

(3) GPU 计算阶段：

(a) GPU 执行核心的矩阵乘法： $S' = Q' \cdot K'^T$ 。

(b) GPU 将结果 S' 发送回 TDX。

(4) 最终计算阶段（TDX 内执行）：

(a) TDX 接收到 S' ，并通过减去交叉项和预计算的校正项来恢复结果：

$$S = S' - Q \cdot M_1^T - M_2 \cdot K^T - M_2 * M_1^T = S' - Q \cdot U_1^T \cdot V_1^T - U_2 \cdot (V_2 \cdot K^T) - M_2 * M_1^T。$$

分析： 核心的、复杂度为 $O(n^2d)$ 的矩阵乘法被卸载到 GPU。TDX 中的计算开销主要与秩 r 相关，例如校正项的乘法复杂度为 $O(nrd)$ 。通过选择一个十以内的 r ，可以使 TDX 的计算开销远低于 GPU，从而实现加速。安全性方面，随机生成的掩码矩阵 M_1 和 M_2 对 GPU 保密，使其难以从 Q' 和 K' 中反推原始信息。

实施例 3：私密向量与动态私密矩阵的乘法

针对大语言模型自回归解码过程中的逐 Token 注意力计算 $\vec{s} = \vec{q} \cdot K^T$ 。这里， $\vec{q}$ 是当前 Token 的私密查询向量，而 K 是包含了所有历史 Token 信息的键缓存，会动态增长，两者都是私密的。

A. 查询计算协议：

假设一个掩码后的键缓存 $K' = K + M_1$ 已存在于 GPU 中。

(1) 离线/一次性预计算阶段（TDX 内执行）：

(a) TDX 生成一个固定的随机矩阵 $H \in \mathbb{R}^{r \times d}$ ，该矩阵在多次查询中保持不变。

(b) TDX 生成一个随机向量池，包含多个一次性的随机向量 $\vec{g} \in \mathbb{R}^{1 \times r}$ 。

(c) TDX 一次性计算出关键的校正项 $C_h = H \cdot K^T$ 。该结果将被后续所有查询重复使用，直到键缓存更新。

(2) 在线查询阶段（TDX 发往 GPU）：

(a) TDX 获得当前 Token 的查询向量 $\vec{q}$ 。

(b) TDX 从向量池中取出一个未使用的 $\vec{g}$ ，计算掩码查询： $\vec{q}' = \vec{q} + \vec{g} \cdot H$ 。

(c) TDX 将 $\vec{q}'$ 发送给 GPU。

(3) GPU 计算阶段：

(a) GPU 接收 $\vec{q}'$ 并与已有的掩码键缓存计算： $\vec{s}' = \vec{q}' \cdot K'^T$ （等于 $(\vec{q} + \vec{g} \cdot H) \cdot (K^T + M_1)$ ）。

(b) GPU 将结果 $\vec{s}'$ 发回 TDX。

(4) 最终计算阶段（TDX 内执行）：

(a) TDX 接收 $\vec{s}'$ ，并进行解掩码：

$$\vec{s} = \vec{s}' - \vec{q}' * M_1 - \vec{g} \cdot C_h$$

(b) 上述公式中，TDX 利用预计算的 C_h 将 $\vec{g} \cdot H \cdot K^T$ 替换为 $\vec{g} \cdot C_h$ ，从而将原本复杂度为 $O(rdn)$ 的计算降为 $O(rn)$ 。

B. 动态键缓存更新机制：

(1) 累积与触发：可信执行环境（TEE）在其内部安全内存中开辟一个缓冲区，用于累积自上次更新以来新生成的键向量。这些键向量被逐行堆叠，形成一个更新矩阵 ΔK 。当该矩阵的行数（即累积的 Token 数量）达到预设的阈值（例如 128）时，批量更新流程被自动触发。

(2) 掩码生成与处理：一旦更新流程被触发，TEE 会生成一套全新的、用于此次更新的低秩随机掩码 ΔM_k 。为了保持计算高效性，该掩码同样由两个更小的随机矩阵相乘生成（ $\Delta M_k = \Delta U_k * \Delta V_k$ ）。随后，TEE 在内部计算掩码后的更新矩阵： $\Delta K' = \Delta K + \Delta M_k$ 。

(3) 卸载与合并：TEE 将计算出的掩码后更新矩阵 $\Delta K'$ 发送至不可信的 GPU。GPU 接收到数据后，将其在行维度上（即 Token 维度）拼接到已存储的掩码键缓存 K' 的末尾，形成新的、尺寸更大的掩码键缓存： $K' = \text{concat}(K', \Delta K')$ 。

(4) 矫正项同步：TEE 更新相关的矫正项（ $C_h = H \cdot K^T + H \cdot \Delta K$ ），以确保后续新的查询请求能够获得正确的计算结果。

分析： 本方案将每个 Token 注意力计算的核心计算 $O(dn)$ 卸载到 GPU。通过预计算和分摊矫正项 C_h 的成本，以及批量更新键缓存，极大地减少了 TDX 在每次查询时的在线计算负担，实现了高效的动态隐私保护计算。安全性方面，尽管 H 是固定的，但每个查询都使用新的一次性随机向量 $\vec{g}$ 生成掩码，保证了 GPU 无法预测或破解每次的查询掩码，从而保护了查询向量 $\vec{q}$ 的隐私。

说明书附图

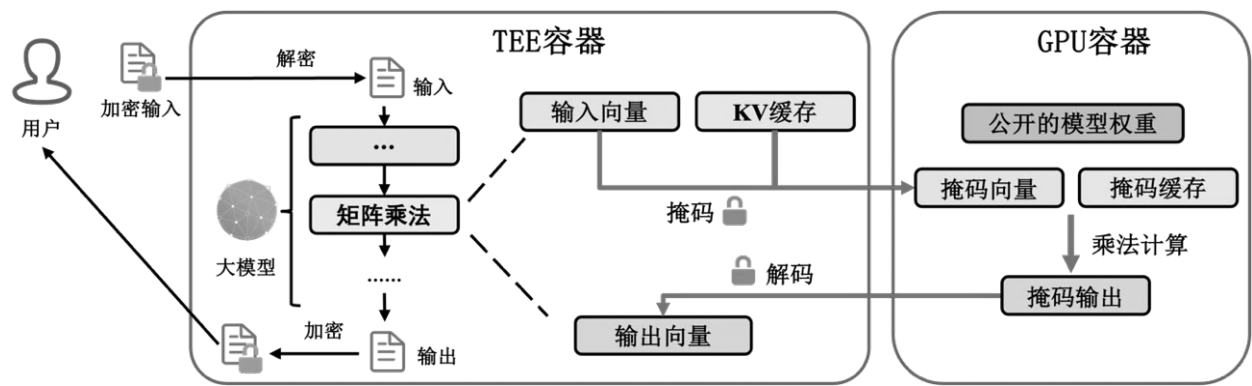


图 1